

MATH 3190 Homework 8

Due April 2, 2026

Focus: Notes 9

Total Points: 90 + 10 (Self-Critique Comment and Uploading to GitHub) = 100

Note that your homework should be completed in Quarto. You can just add your answers and code to this document in the appropriate part. The file should be rendered as an html document or pdf document (html is quicker to render and you will need an installation of LaTeX to render as a pdf). You will “turn in” this homework by uploading the files, the .qmd and output file, to your GitHub `Math_3190_Assignment` repository in the `Homework/Homework_8` directory. Make sure all supporting files (if there are any) are in this directory as well. You should use `git` and `Unix` commands in a terminal for this (i.e. don’t manually upload files to GitHub). In addition, you will submit just your html or pdf file on Canvas to unlock the solutions.

Don’t forget to also put a self-critique comment on Canvas indicating what you did correctly and incorrectly and include any questions you may have about the assignment after submitting the assignment and thoroughly looking through the solutions.

Problem 1 (10 points)

Three airlines serve a small town in Ohio. Airline A has 52% of all scheduled flights, airline B has 35% and airline C has the remaining 13%. Their on-time rates are 85%, 67%, and 41%, respectively. A flight just left on-time. What is the probability that it was a flight of airline A?

Problem 2 (15 points)

Suppose we have a data set with each observation x_i independent and identically exponentially distributed for $i = 1, 2, \dots, n$. That is, $x_i \sim \text{Exp}(\lambda)$ where λ is the rate parameter. We would like to find a posterior (or at least a function proportional to it) for λ .

Part a (6 points)

Write down the likelihood function (or a function proportional to it) in this situation. We would call this $p(x|\lambda)$.

Part b (6 points)

Now let λ have a normal prior with mean 0.1 and variance 1: $\lambda \sim N(1/10, 1)$. Use this and the likelihood from part (a) to write down a function that is proportional to the posterior of λ given x . We call this $p(\lambda|x)$.

Part c (3 points)

Which would be more appropriate here to obtain samples of λ , the Gibbs or Metropolis algorithm? Explain why. You may want to look on page 171 of Notes 9 in the conjugate prior table.

Problem 3 (30 points)

Suppose we have the vector $\mathbf{x} = \mathbf{c}(1.83, 1.72, 2.13, 2.49, 0.90, 2.01, 1.51, 3.12, 1.29, 1.54, 2.94, 3.02, 0.93, 2.78)$ that we believe comes from a gamma distribution with shape of 10 and some rate β : $x_i \sim \text{Gam}(10, \beta)$. We will use sampling to obtain some information about β . Let's put a gamma prior on β with a shape of α_0 and a rate of 1: $\beta \sim \text{Gam}(\alpha_0, 1)$.

Part a (5 points)

Use the fact that this is a conjugate prior to write down what kind of distribution the posterior of β , which is $p(\beta|x)$, is.

Part b (5 points)

Let $\alpha_0 = 1$. In an **R** code chunk, sample 10,000 β values from the distribution you wrote down in part (a) using the `rgamma()` function and report the 95% credible interval for β using the 2.5th and 97.5th percentiles.

Part c (3 points)

Repeat part (b) with $\alpha_0 = 10$.

Part d (3 points)

Repeat part (b) with $\alpha_0 = 100$.

Part e (4 points)

Comment on the effect the value of α_0 has on the credible intervals for β as revealed in parts (b), (c), and (d).

Part f (6 points)

Now suppose we have twice as much data (given in the **R** code chunk below). Repeat parts (b), (c), and (d) using this **x** vector given below instead of the other one and report the three 95% credible intervals. Note, this new vector **x** will change the shape and rate parameters used in the `rgamma()` functions.

```
x <- c(1.83, 1.72, 2.13, 2.49, 0.90, 2.01, 1.51, 3.12, 1.29, 1.54,  
       2.94, 3.02, 0.93, 2.78, 2.76, 1.70, 1.42, 2.16, 1.07, 2.21,  
       2.38, 2.27, 1.72, 1.44, 1.54, 1.72, 1.87, 1.39)
```

Part g (4 points)

In this problem, the true β value is 5. Write a sentence or two about the effect adding more data has to these credible intervals by comparing the intervals from parts (b)-(d) to the intervals from part (f).

Problem 4 (35 points)

Let's apply the Bayesian framework to a regression problem. In the `math3190package` library is the `treeseeds` dataset (it is also on GitHub) that contains information about species of tree, the count of seeds it produces, and the average weight of those seeds in mg.

Part a (3 points)

Using the `treeseeds` data, create new variables defined by taking the log of the counts and weights. Fit an OLS regression model using logged weight to predict logged count.

```
library(math3190package) |> suppressPackageStartupMessages()
tree <- treeseeds |>
  mutate(lcount = log(count),
         lweight = log(weight))

# Fit model here
```

Part b (5 points)

Please read this carefully and try to understand what is being done here. There is just one question at the end of this part for you. We will walk through the mathematics and code of obtaining the posterior and samples together here since this problem will focus on looking at diagnostics, but I want you to see the **Metropolis algorithm** in action! Make sure to read this in the html or pdf file where all equations are rendered nicely.

Assuming the true errors are normal with mean 0 and variance σ^2 , $\epsilon_i \sim N(0, \sigma^2)$, it can be shown that each y_i has the distribution

$$p(y_i|x_i, \beta_0, \beta_1, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_i - \beta_0 - \beta_1 x_i)^2\right)$$

So, we can write the likelihood is

$$p(y_i|x_i, \beta_0, \beta_1, \sigma^2) \propto \exp\left(-\frac{1}{2\sigma^2}(y_i - \beta_0 - \beta_1 x_i)^2\right)$$

where y_i is the `log(count)` for observation i and x_i is the `log(weight)` for observation i . Note that here we think of y as being random and x as being fixed. We could, in theory, think of the vector x as also being random and put a prior on it. But we won't do that here.

Now, let's just put uniform priors on β_0 and β_1 so the priors are proportional to 1. Also, let's assume $\sigma^2 = 1$. This seems reasonable since s_e^2 , the MSE, is 0.877. Of course, we could put a prior on σ^2 as well and sample it too, but we will focus on only sampling β_0 and β_1 .

Now, with those uniform priors, and plugging in 1 for σ^2 , we have that the joint posterior of β_0 and β_1 is:

$$p(\beta_0, \beta_1|x, y) \propto \exp\left(-\frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2\right) = f(\beta_0, \beta_1|x, y).$$

Then, we can take the log to get

$$\ln(f(\beta_0, \beta_1 | x, y)) = -\frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2.$$

Our goal now is to obtain samples of β_0 and β_1 . Let's use the Metropolis algorithm to do this. Using the log of the function proportional to the joint posterior of β_0 and β_1 , $\ln(f(\beta_0, \beta_1 | x, y))$, we'll write a Metropolis algorithm in **R**. For β_0 , we will use a normal proposal distribution centered at the previous value, $\beta_0^{(i)}$, with a standard deviation of 0.8 and for β_1 , and we will use a normal proposal distribution centered at the previous value, $\beta_1^{(i)}$, with a standard deviation of 0.1. The starting values don't matter too much, but we'll use $\beta_0^{(0)} = 10$ and $\beta_1^{(0)} = -0.5$.

This code will generate 10,000 samples and plot the chains for β_0 and β_1 . In addition, the 95% credible interval for β_0 and β_1 are given.

```
library(patchwork)
set.seed(2026)
nsamps <- 10000
beta0 <- rep(0, nsamps)
beta1 <- rep(0, nsamps)
beta0[1] <- 10
beta1[1] <- -0.5
for (i in 1:(nsamps - 1)) {
  beta0_star <- rnorm(1, beta0[i], 0.8) # Proposed new value for intercept
  beta1_star <- rnorm(1, beta1[i], 0.1) # Proposed new value for slope
  # y is tree$lcount and x is tree$lweight
  lf1 <- -1 / 2 * sum((tree$lcount - beta0_star - beta1_star * tree$lweight)^2)
  lf2 <- -1 / 2 * sum((tree$lcount - beta0[i] - beta1[i] * tree$lweight)^2)
  if (log(runif(1)) < lf1 - lf2) {
    # Accept the proposed values as new sample values if the above
    # condition is satisfied
    beta0[i + 1] <- beta0_star
    beta1[i + 1] <- beta1_star
  } else {
    # Otherwise set the new sample values to be equal to the previous ones
    beta0[i + 1] <- beta0[i]
    beta1[i + 1] <- beta1[i]
  }
}

mcmc_df <- data.frame(index = 1:nsamps, beta0 = beta0, beta1 = beta1)
mcmc_plot1 <-
```

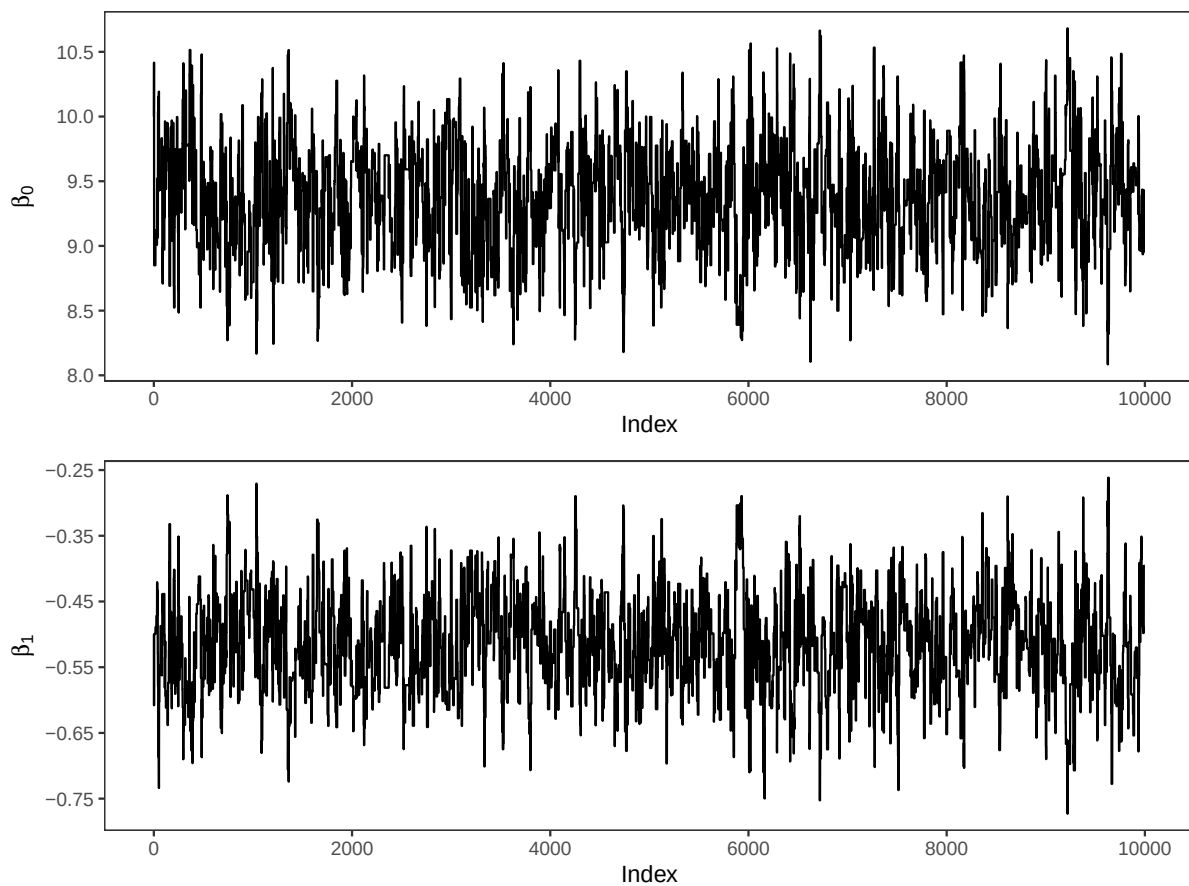
```

ggplot(mcmc_df, aes(x = index, y = beta0)) +
  geom_line() +
  theme_test() +
  scale_x_continuous(breaks = seq(0, 10000, by = 2000)) +
  scale_y_continuous(breaks = seq(8, 11, by = 0.5)) +
  labs(x = "Index", y = expression(beta[0]))

mcmc_plot2 <-
  ggplot(mcmc_df, aes(x = index, y = beta1)) +
  geom_line() +
  theme_test() +
  scale_x_continuous(breaks = seq(0, 10000, by = 2000)) +
  scale_y_continuous(breaks = seq(-0.75, -0.25, by = 0.1)) +
  labs(x = "Index", y = expression(beta[1]))

mcmc_plot1 / mcmc_plot2

```



```
quantile(beta0, c(.025, .975))
```

```
      2.5%      97.5%  
8.526416 10.210557
```

```
quantile(beta1, c(.025, .975))
```

```
      2.5%      97.5%  
-0.6606829 -0.3681291
```

The plots of the chains are given in the output above.

The 95% credible interval for β_0 is (8.53, 10.21).

The 95% credible interval for β_1 is (−0.661, −0.368).

Your intervals may differ a bit due to the random sampling.

Okay, here is the question for you! Based on the plots of the chains above, does it look like the Metropolis sampling worked fairly well? Explain.

Part c (3 points)

Interpret both of the credible intervals from part (b).

Part d (5 points)

Find and report the integrated autocorrelation time for the β_0 and β_1 chains. Each chain will have their own $\hat{\tau}_{int}$ value, so you should report two (although they will be similar).

Part e (3 points)

Based on the integrated autocorrelation time for the β_0 and β_1 chains, how many MCMC samples would you need to generate to get the equivalent of 10,000 independent samples?

Part f (3 points)

Let's compare these credible intervals to some other intervals. First, obtain the 95% t confidence intervals for β_0 and β_1 just using the `confint()` function on the model you fit in part (a) and report them here.

Part g (8 points)

Now let's obtain confidence intervals using bootstrapping the cases like we did with in Notes 7 and HW 5. You can use the `bootstrapping()` function in the `math3190package` for this or do it yourself with a loop. Set a seed and then using at least 10,000 bootstrap samples, report the 95% percentile confidence intervals for β_0 and β_1 .

Part i (5 points)

Write a couple sentences comparing all of the intervals in parts (b), (f), and (g).